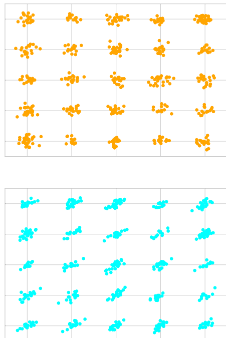
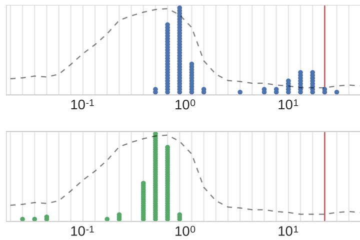
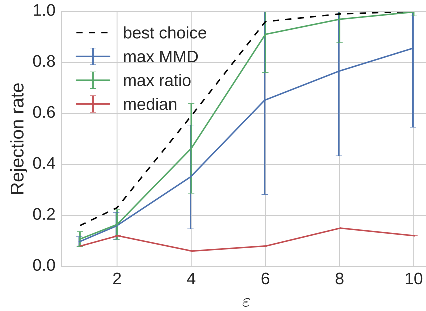


(a) Samples of size 500 with $\varepsilon = 6$ from P (top) and Q (bottom).



(b) Bandwidths chosen by maximizing $\widehat{\mathrm{MMD}}_{\mathrm{U}}^2$ (top, blue) and $\hat{t}$ (bottom, green), as well as the median heuristic (red), for $\varepsilon = 6$. Gray lines show the power of each bandwidth: $\sigma = 0.67$ had power 96%, $\sigma = 10$ had 10%.



(c) Mean and standard deviation of rejection rate as ε increases. "Best choice" shows the mean power of the bandwidth with highest rejection rate for each problem.

Figure 2: Results for the Blobs problem. Maximizing $\hat{t}$ performs near-optimally.

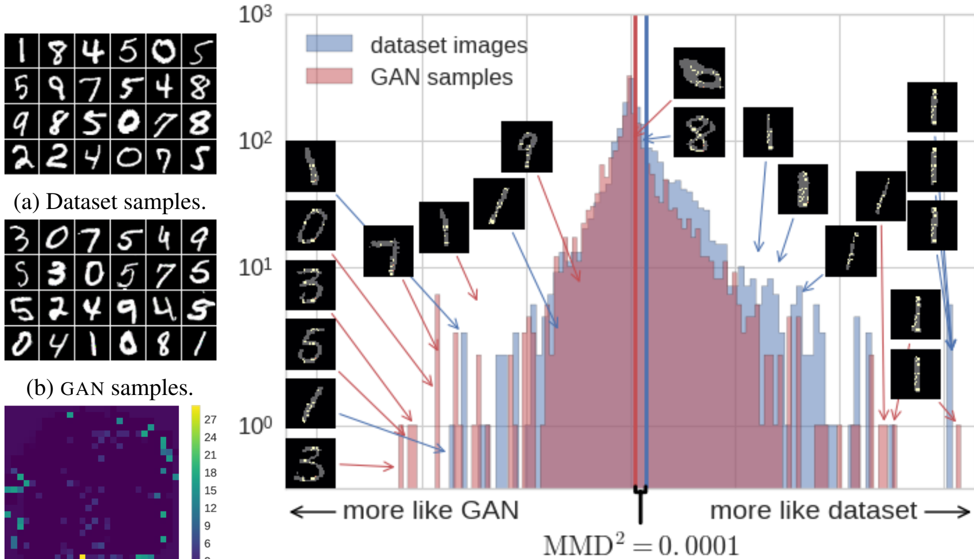
only in 52.4% of cases. Comparing the results, however, there are several pixel-level artifacts that make distinguishing the datasets trivial; our methods can pick up on these quickly.

To make the problem more interesting, we discretized the sampled pixels into black or white (which barely changes the images visually). The samples are then in $\{0,1\}^{28\times 28}$. We trained an automatic relevance determination (ARD)-type kernel: in the notation of Section 2.1, z scales each pixel by some learned value, and k is a Gaussian RBF kernel with a learned global bandwidth. We optimized $\hat{t}$ on 2 000 samples in batches of size 500 using the Adam optimizer (Kingma & Ba, 2015), where the learned weights are visualized in Figure 3c. This network has essentially perfect discriminative power: testing it on 100 different samples with 1000 permutations for each test, in 98 cases we obtained p-values of 0.000 and twice got 0.001. By contrast, using an RBF kernel with a bandwidth optimized by maximizing the t statistic gave a less powerful test: the worst p-value in 100 repetitions was 0.135, with power 57% at the $\alpha = 0.01$ threshold. An RBF kernel based on the median heuristic, which here found a bandwidth five times the size of the t-statistic-optimized bandwidth, performed worse still: three out of 100 repetitions found a p-value of exactly 1.000, and power at the .01 threshold was 42%. The learned weights show that the model differs from the true dataset along the outsides of images, as well as along a vertical line in the center.

We can investigate these results in further detail using the approach of Lloyd & Ghahramani (2015), considering the witness function associated with the MMD, which has largest amplitude where the probability mass of the two samples is most different. Thus, samples falling at maxima and minima of the witness function best represent the difference in the distributions. The value of the witness function on each sample is plotted in Figure 3d, along with some images with different values of the witness function. Apparently, the GAN is slightly overproducing images resembling the `/`-like digits on the left, while underproducing vertical `1`s. It is not the case that the GAN is simply underproducing `1`s in general: the p-values of a χ^2 contingency test between the outputs of digit classifiers on the two distributions are uniform. This subtle difference in proportions among types of digits would be quite difficult for human observers to detect. Our testing framework allows the model developer to find such differences and decide whether to act on them. One could use a more complex representation function z to detect even more subtle differences between distributions.

GAN criterion We now demonstrate the use of MMD as a training criterion in GANs. We consider two basic approaches, and train on MNIST.[6] First, the generative moment matching network (GMMN; Figure 4a) approach (Li et al., 2015; Dziugaite et al., 2015) uses an MMD statistic computed with an

[6] **Implementation details:** We used the architecture of Li et al. (2015): the generator consists of fully connected layers with sizes $10, 64, 256, 256, 1024, 784$, each with ReLU activations except the last, which uses sigmoids. The kernel function for GMMNs is a sum of Gaussian RBF kernels with fixed bandwidths $2, 5, 10, 20, 40, 80$. For the feature matching GAN, we use a discriminator with fully connected layers of size $512, 256, 256, 128, 64$, each with sigmoid activation. We then concatenate the raw image and each layer of features as input to the same mixture of RBF kernels as for GMMNs. We optimize with SGD. Initialization for all parameters are Gaussian with standard deviation 0.1 for the GMMNs and 0.2 for feature matching. Learning rates



(a) Dataset samples.

(b) GAN samples.

(c) ARD weights.

(d) The witness function $\mathbb{E}_{x\sim P}\, k(x,\cdot) - \mathbb{E}_{y\sim Q}\, k(y,\cdot)$ evaluated on a test set. Images are shown with ARD weights highlighted. Vertical lines show distribution means; the distance between them is small but highly consistent.

Figure 3: Model criticism of Salimans et al. (2016)'s semi-supervised GAN on MNIST.

RBF kernel directly on the images as the discriminator of a GAN model. The t-GMMN (Figure 4b) has the generator minimize the $\hat{t}_k$ statistic for a fixed kernel.[7] Compared to standard GMMNs, the t-GMMN more directly attempts to make the distributions *indistinguishable* under the kernel function; it avoids a situation like that of Figure 3d, where although the MMD value is quite small, the two distributions are perfectly distinguishable due to the small variance.

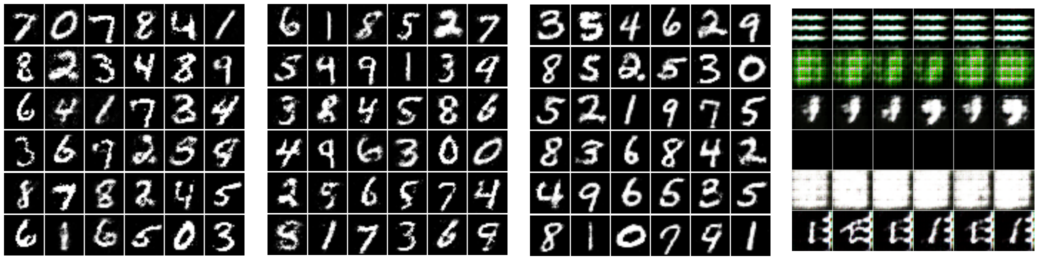
Next, feature matching GANs (Figure 4c) train the discriminator as a classifier like a normal GAN, but train the generator to minimize the MMD between generator samples and reference samples with a kernel computed on intermediate features of the discriminator. Salimans et al. (2016) proposed feature matching using the mean features at the top of the discriminator (effectively using an MMD with a linear kernel); we instead use MMD with a mixture of RBF kernels, ensuring that the full feature distributions match, rather than just their means. This helps avoid the common failure mode of GANs where the generator collapses to outputting a small number of samples considered highly realistic by the discriminator. Using the MMD-based approach, however, no single point can approximate the feature *distribution*. The minibatch discrimination approach of Salimans et al. (2016) attempts to solve the same problem, by introducing features measuring the similarity of each sample to a selection of other samples, but we were unable to get it to work without labels to force the discriminator in a reasonable direction; Figure 4d demonstrates some of those failures, with each row showing six samples from each of six representative runs of the model.

Acknowledgements

We would like to thank Tim Salimans, Ian Goodfellow, and Wojciech Zaremba for providing their code and for gracious assistance in using it, as well as Jeff Schneider for helpful discussions.

are $2, 0.02, 0.5$, respectively. Learning rate for the feature matching discriminator is set to 0.01. All experiments are run for $50\,000$ iterations and use a momentum optimizer with with momentum 0.9.

[7] One could additionally update the kernel adversarially, by maximizing the $\hat{t}_k$ statistic based on generator samples, but we had difficulty in optimizing this model: the interplay between generator and discriminator adds some difficulty to this task.



(a) GMMN (b) t-GMMN (c) Feature matching (d) Improved GAN

Figure 4: MNIST digits from various models. Part d shows six runs of the minibatch discrimination model of Salimans et al. (2016), trained without labels — the same model that, with labels, generated Figure 3b. (The third row is the closest we got the model to generating digits without any labels.)

References

V. Alba Fernández, M. Jiménez-Gamero, and J. Muñoz Garcia. A test for the two-sample problem based on empirical characteristic functions. *Computational Statistics & Data Analysis*, 52:3730–3748, 2008.

Wacha Bounliphone, Eugene Belilovsky, Matthew B. Blaschko, Ioannis Antonoglou, and Arthur Gretton. A test of relative similarity for model selection in generative models. In *International Conference on Learning Representations*, 2016. arXiv:1511.04581.

Olivier Breuleux, Yoshua Bengio, and Pascal Vincent. Quickly generating representative samples from an rbm-derived process. *Neural Computation*, 23(8):2053–2073, 2011.

Emily Denton, Soumith Chintala, Arthur Szlam, and Rob Fergus. Deep generative image models using a laplacian pyramid of adversarial networks. In *Advances in Neural Information Processing Systems*, pp. 1486–1494. 2015.

Gintare Karolina Dziugaite, Daniel M. Roy, and Zoubin Ghahramani. Training generative neural networks via maximum mean discrepancy optimization. In *Uncertainty in Artificial Intelligence*, 2015. arXiv:1505.03906.

Kenji Fukumizu, Arthur Gretton, Xiaohai Sun, and Bernhard Schölkopf. Kernel measures of conditional dependence. In *Advances in Neural Information Processing Systems*, 2008.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014. arXiv:1406.2661.

Arthur Gretton, Kenji Fukumizu, Zaid Harchaoui, and Bharath K. Sriperumbudur. A fast, consistent kernel two-sample test. In *Advances in Neural Information Processing Systems*, pp. 673–681, 2009.

Arthur Gretton, Karsten M. Borgwardt, Malte J Rasch, Bernhard Schölkopf, and Alex J. Smola. A kernel two-sample test. *The Journal of Machine Learning Research*, 13, 2012a.

Arthur Gretton, Bharath Sriperumbudur, Dino Sejdinovic, Heiko Strathmann, and Massimiliano Pontil. Optimal kernel choice for large-scale two-sample tests. In *Advances in Neural Information Processing Systems*, 2012b.

Gaël Guennebaud, Benoît Jacob, et al. Eigen v3. http://eigen.tuxfamily.org, 2010.

Michael U. Gutmann, Ritabrata Dutta, Samuel Kaski, and Jukka Corander. Statistical inference of intractable generative models via classification. 2014. arXiv:1407.4981.

Ferenc Huszar. How (not) to train your generative model: Scheduled sampling, likelihood, adversary? 2015. arXiv:1511.05101.